Claude Code
Tutorial da segunda aula

Rules, agents, skills e planejamento na prática

Guia do aluno • Aula de 14 de outubro de 2026 • Beto / Dev em Dobro

Você vai organizar as instruções da landing page da academia, criar um agente que explica código, testar uma skill de segurança e planejar um gerador de orçamento para freelancers. Siga as etapas em ordem e copie os blocos no destino indicado.

Antes de começar

Tenha o Claude Code instalado e autenticado, um editor como o Cursor e o projeto da primeira aula: a pasta lp-claude-code com index.html, CLAUDE.md e a referência visual da academia. Se ainda não fez a primeira aula, prepare esse projeto antes dos testes de alteração da landing.

Como usar os blocos

CONVERSA: envie como mensagem ao Claude Code. TERMINAL: execute no terminal do editor. ARQUIVO: cole no arquivo indicado, sem incluir o nome do caminho no conteúdo. Preserve as quebras de linha e a indentação. Nos arquivos com cabeçalho entre --- , o primeiro --- deve estar na primeira linha.

Terminal - conferir a versão

```
claude --version
```

Comandos e rótulos podem variar entre versões. Se um recurso não aparecer, confira a ajuda da instalação. A aula cita recursos recentes; não confunda um comando ausente com erro no conteúdo do seu projeto.

Percurso

1. Testar o CLAUDE.md
2. Conhecer a pasta .claude
3. Ver as rules de exemplo
4. Reorganizar as regras
5. Criar e testar agents
6. Criar e testar a skill
7. Planejar o gerador de orçamento
8. Executar a primeira etapa com /goal
9. Conferir a prática e os comandos de apoio

1 Testar o CLAUDE.md

Abra o projeto da academia no editor. O CLAUDE.md contém as orientações gerais do projeto. Acrescente o trecho abaixo ao final do arquivo existente, sem apagar as instruções da primeira aula.

Arquivo - CLAUDE.md, no final

```
## Teste de leitura (temporário)

- Termine TODA resposta com a linha exata: `Regras do projeto aplicadas:` seguida dos títulos das regras deste arquivo que você usou na resposta.
- Se nenhuma regra se aplicou, escreva: `Regras do projeto aplicadas: nenhuma`.
```

Salve, encerre a conversa e abra uma nova sessão do Claude Code na mesma pasta. Confira o contexto carregado:

```
/context
```

Conversa - alterar o botão

```
Troca o texto do botão principal da landing para "Quero minha aula experimental".
```

Conversa - verificar as regras aplicadas

```
Quais regras do CLAUDE.md (e de .claude/rules, se houver) você seguiu nessa última alteração? Pra cada regra, cite o texto dela e me mostre o arquivo e o trecho do código onde dá pra ver que ela foi seguida. Se alguma regra não se aplicou, diga qual e por quê.
```

Confira: abra index.html e veja a página no navegador. O botão deve ter o texto solicitado. Compare os trechos citados pelo Claude com o código. A marca “Regras do projeto aplicadas” é um sinal de uso das instruções, mas não comprova sozinha que todas foram obedecidas. Mantenha a regra de teste até terminar a reorganização.

2 Conhecer a pasta .claude

Procure .claude na árvore do editor. Se não existir, crie essa pasta dentro do projeto. Tire um print da estrutura real e anexe à conversa junto com o prompt abaixo. Ele cria um arquivo HTML explicativo, que você abre no navegador.

Conversa - enviar junto com o print

```
Esse print mostra a estrutura da pasta .claude do meu projeto. Crie um arquivo chamado
estrutura-claude.html, que eu possa abrir no navegador, com um infográfico explicativo dessa estrutura
para alguém que nunca programou:

- Desenhe a árvore de pastas e arquivos exatamente como está no print (não invente pastas que não
aparecem).
- Ao lado de cada item, escreva em uma frase, em português simples, para que ele serve e quando o Claude
Code lê esse item.
- Use uma cor diferente para cada tipo: CLAUDE.md, settings, rules, agents e skills.
- Inclua uma legenda curta no rodapé explicando a diferença entre a .claude do projeto e a ~/.claude do
usuário.
- Tudo em um único arquivo HTML, sem depender de internet.
Se não tiver certeza do que algum item faz, consulte a documentação oficial do Claude Code
(code.claude.com/docs/en/claude-directory) antes de escrever.
```

Abra estrutura-claude.html e compare o desenho com o print. Depois, crie dentro de .claude as pastas rules, agents e skills, com os nomes em minúsculas. Não precisa pedir isso à IA: use a opção de criar pasta do editor.

```
lp-claude-code/
  index.html
  CLAUDE.md
  .claude/
    rules/
    agents/
    skills/
```

Reinicie a sessão depois de criar as pastas. As rules organizam orientações por assunto; os agents recebem tarefas especializadas; as skills guardam procedimentos reutilizáveis.

3 Conhecer as rules de exemplo

A aula mostra três exemplos. Para reproduzi-los, crie cada arquivo separadamente no caminho indicado. O campo paths restringe o assunto da terceira regra aos arquivos HTML.

Arquivo - .claude/rules/idioma-e-tom.md

```
# Idioma e tom

- Responda sempre em português do Brasil.
- Todo texto visível da página (títulos, botões, mensagens de erro) deve estar em português do Brasil.
- Explique termos técnicos em uma frase simples na primeira vez que aparecerem.
```

Arquivo - .claude/rules/seguranca.md

```
# Segurança

- Nunca escreva senhas, tokens ou chaves de API dentro do código. Use variáveis de ambiente e um arquivo
.env que não vai para o Git.
- Nunca apague arquivos ou pastas sem pedir confirmação antes.
- Links que abrem em nova aba usam rel="noopener noreferrer".
```

Arquivo - .claude/rules/html-acessibilidade.md

```
---
paths:
  - "**/*.html"
---

# Acessibilidade no HTML

- Toda imagem tem atributo alt descrevendo o conteúdo.
- Botões e links têm texto que diz o que fazem (nada de "clique aqui").
- Use um único h1 por página e siga a ordem h2, h3.
```

Confira os nomes dos arquivos e a indentação do campo paths. Essas orientações são exemplos da aula; leia o conteúdo antes de incorporá-lo ao seu projeto.

4 Reorganizar o CLAUDE.md

Na conversa do projeto da academia, peça a separação das instruções por assunto. A intenção é preservar o conteúdo e facilitar a manutenção.

Conversa - dividir as instruções em rules

```
Quero reorganizar as instruções deste projeto. Pegue o nosso CLAUDE.md e separe cada assunto em um arquivo de regra próprio dentro de .claude/rules/, um arquivo .md por assunto, com nome descritivo em minúsculas e com hífen.

Siga a especificação oficial de rules do Claude Code. Se você não tiver certeza de como ela funciona (onde os arquivos ficam, como funciona o frontmatter e o campo paths), consulte a documentação oficial antes de criar qualquer arquivo: https://code.claude.com/docs/en/memory

Regras para essa divisão:
- Não mude o sentido de nenhuma instrução e não invente regras novas.
- Se uma regra só faz sentido para certos arquivos (por exemplo, só para HTML), use paths no frontmatter. Se vale para tudo, não coloque frontmatter.
- Deixe no CLAUDE.md só uma descrição curta do projeto, em poucas linhas.
- No final, me mostre uma lista de cada arquivo criado, com uma frase explicando o que ele cobre, e confirme que nenhuma instrução do CLAUDE.md original ficou de fora.
```

Leia cada arquivo criado e compare com o CLAUDE.md anterior. Os próximos blocos são a referência apresentada na aula. Seu resultado pode variar conforme o conteúdo original; não substitua regras particulares apenas para igualar os exemplos.

Referência - CLAUDE.md enxuto

```
# Recriação de Design de Site

Projeto de recriação fiel de landing pages a partir de uma imagem ou site de referência. As regras detalhadas estão em .claude/rules/.
```

Referência - .claude/rules/fidelidade-a-referencia.md

```
# Fidelidade à referência

- Seja fiel à referência: nada de "melhorar" o design por conta própria.
- Não inclua seções, recursos ou conteúdos que não aparecem na imagem de referência.
- Se o usuário passar classes CSS ou tokens de estilo, aplique exatamente como vieram.
- Ao comparar prints, aponte o problema com precisão (ex.: "o espaçamento entre os cards está em 16px, mas deveria ser 24px", "o título está com 32px e na referência tem 24px").
```

Referência - .claude/rules/padroes-tecnicos.md

```
---
paths:
  - "**/*.html"
---

# Padrões técnicos

- Comece pelo mobile e garanta que o layout seja responsivo.
- Carregue o Tailwind CSS pela CDN (`<script src="https://cdn.tailwindcss.com"></script>`).
- Se as imagens originais não forem enviadas, use placeholders de `https://placehold.co/`.
- Mantenha o código organizado, sem abstrair demais: classes do Tailwind direto no HTML estão ok.
```

Referência - .claude/rules/fluxo-de-verificacao-visual.md

```
# Fluxo de trabalho e verificação visual

- Entregue tudo em um único `index.html`, a menos que o usuário peça outra estrutura.

1. **Crie** um arquivo `index.html` com Tailwind CSS (via CDN), com todo o conteúdo inline. Use arquivos externos só se o usuário pedir.
2. **Tire um print** da página renderizada com Puppeteer (`npx puppeteer screenshot index.html --fullpage` ou algo equivalente). Se a página tiver seções bem definidas, faça prints separados de cada uma também.
3. **Compare** o print com a imagem de referência e procure diferenças em: cores (valores hex exatos); tamanho, peso e altura de linha das fontes; espaçamentos e paddings (em px); alinhamento e posicionamento; tamanho e posição de imagens e ícones; arredondamento de bordas, sombras e efeitos; comportamento responsivo.
4. **Corrija** cada diferença encontrada, editando o HTML e as classes do Tailwind.
5. **Tire um novo print** e compare de novo.
6. **Repita** os passos 3 a 5 até que tudo esteja a no máximo ~2–3px da referência.

Nunca pare na primeira rodada. Faça sempre pelo menos 2 comparações. Só encerre quando o usuário mandar ou quando não houver mais nenhuma diferença visível.
```

Referência - .claude/rules/teste-de-leitura.md

```
# Teste de leitura (temporário)

- Termine TODA resposta com a linha exata: `Regras do projeto aplicadas:` seguida dos nomes das regras que você usou na resposta.
- Se nenhuma regra se aplicou, escreva: `Regras do projeto aplicadas: nenhuma`.
```

Reinicie a sessão e execute /context. Regras condicionadas por paths são relevantes quando você trabalha com os arquivos correspondentes. Faça o segundo teste:

Conversa - testar depois da reorganização

```
Troca o título da seção de planos para "Escolha o seu plano".
```

Confira: o título da seção de planos deve mudar. Observe a marca de teste e reutilize o prompt de verificação da etapa 1 para conferir o código. A reorganização só está concluída quando nenhuma instrução original ficou de fora.

5 Criar o agente explicador de código

Crie o arquivo abaixo dentro de .claude/agents. O cabeçalho informa o nome, a finalidade e as ferramentas; o corpo define como explicar o código.

Arquivo - .claude/agents/explicador-de-codigo.md

```
---
name: explicador-de-codigo
description: Explica arquivos e trechos de código do projeto para quem está começando a programar, em
português simples. Use quando o usuário pedir para entender o que um código faz.
tools: Read, Grep, Glob
---

Você é um professor paciente que explica código para pessoas que nunca programaram.

Quando receber um pedido:
1. Leia o arquivo ou trecho indicado antes de explicar. Se o pedido não disser qual arquivo, procure no
projeto o mais provável e diga qual você escolheu.
2. Responda sempre neste formato:
   - **Em uma frase:** o que esse código faz, sem nenhum termo técnico.
   - **Passo a passo:** explique as partes principais na ordem em que aparecem, citando o arquivo e as
linhas.
   - **Comparação do dia a dia:** uma analogia simples que ajude a entender.
   - **Termo técnico do dia:** escolha um termo que apareceu no código e explique em uma frase.
3. Use frases curtas, em português do Brasil, e trate a pessoa por "você".
4. Nunca altere arquivos. Se perceber um erro no código, apenas avise no final, em uma linha.
```

Conversa - testar o agente

```
Use o agente explicador-de-codigo pra me explicar o que faz o cabeçalho (o topo com o menu) do index.html
da landing da academia.
```

A pauta também apresenta duas alternativas de menção. Use uma por vez, caso sua interface ofereça esse formato. O pedido por nome acima evita depender do seletor de menções.

Conversa - alternativa pelo seletor de menções

```
@"explicador-de-codigo (agent)" me explica o que faz o cabeçalho do index.html
```

Conversa - alternativa digitada

```
@agent-explicador-de-codigo me explica o que faz o cabeçalho do index.html
```

Confira: veja se houve delegação ao agente e se a resposta inclui uma frase de resumo, passo a passo com linhas do arquivo, comparação do dia a dia e um termo técnico explicado. Nenhum arquivo deve ser alterado.

5A Outros agents mostrados na aula

Estes exemplos foram apresentados para ilustrar outras especialidades. São opcionais para a prática do explicador, mas seus conteúdos completos estão aqui para você reproduzir.

Arquivo - .claude/agents/revisor-de-textos.md

```
---
name: revisor-de-textos
description: Revisa os textos visíveis da landing (títulos, botões, chamadas) e aponta erros de português
e frases confusas. Use quando o usuário pedir revisão de texto ou copy.
tools: Read, Grep, Glob
---

Você é um revisor de textos de páginas de venda. Leia só os textos que o visitante vê. Para cada
problema, mostre o texto atual, o texto sugerido e o motivo em uma frase. Não altere arquivos.
```

Arquivo - .claude/agents/testador-responsivo.md

```
---
name: testador-responsivo
description: Confere se a página funciona bem em celular, tablet e desktop. Use depois de mudanças de
layout.
---

Você testa layout responsivo. Verifique a página nas larguras de 375px, 768px e 1280px e liste o que
quebra em cada uma, do mais grave para o menos grave.
```

O teste responsivo depende de acesso à página renderizada. Confira evidências nas três larguras; ler o HTML não comprova o comportamento visual.

6 Criar a skill de segurança

Em .claude/skills, crie a pasta checklist-seguranca-web e, dentro dela, SKILL.md. Copie o conteúdo completo a seguir, começando pelo cabeçalho.

Arquivo - .claude/skills/checklist-seguranca-web/SKILL.md

```
---
name: checklist-seguranca-web
description: Revisa páginas web (HTML, CSS, JavaScript) em busca de problemas de segurança comuns em projetos de iniciante e entrega um relatório com gravidade e correção. Use quando o usuário perguntar se a página está segura, pedir revisão de segurança ou antes de publicar um site.
allowed-tools: Read Grep Glob
disallowed-tools: Edit Write NotebookEdit Bash
---

# Checklist de segurança web

Você vai revisar o projeto e gerar um relatório. Não altere nenhum arquivo: só aponte os problemas e como corrigir.

## O que verificar

1. **Segredos expostos:** chaves de API, tokens, senhas ou chaves do Stripe escritas direto no código (procure por "key", "token", "secret", "password", "sk_live", "sk_test").
2. **Links em nova aba:** todo link com target="_blank" precisa de rel="noopener noreferrer".
3. **Scripts de fora:** scripts carregados de outros sites (CDN). Liste cada um e diga de onde vem, para o usuário saber em quem está confiando.
4. **Formulários:** campos sem validação (required, type="email", limites de tamanho) e formulários que enviam dados para endereços com http:// em vez de https://.
5. **Conteúdo inserido no HTML via JavaScript:** uso de innerHTML com dados digitados pelo usuário, que pode permitir que alguém injete código na página.
6. **Endereços inseguros:** imagens, scripts ou links com http:// em vez de https://.
7. **Arquivos que não deveriam ir para o ar:** arquivos .env, backups e anotações com dados pessoais dentro da pasta do projeto.

## Formato do relatório

Comece com uma tabela com as colunas: Gravidade (Alta, Média ou Baixa), Problema, Arquivo e linha, Como corrigir.

Depois da tabela, escreva um **Resumo** de no máximo três frases, em português simples, dizendo se a página está pronta para publicar ou o que precisa ser resolvido antes.

Se não encontrar problemas em um item, não invente: escreva "Nada encontrado" para esse item no final do relatório.
```

6A Testar a skill de segurança

Faça os dois testes abaixo no projeto da academia. O primeiro chama a skill pelo nome; o segundo verifica se o Claude associa uma pergunta comum ao procedimento criado.

Conversa - chamar a skill diretamente

```
/checklist-seguranca-web
```

Em uma conversa nova do mesmo projeto, teste também a escolha por linguagem natural:

Conversa - testar a escolha automática

```
Essa landing da academia está segura pra eu publicar e mandar pro cliente?
```

Confira: observe o registro de uso da skill e a tabela com gravidade, problema, arquivo e linha e correção, seguida do resumo. Verifique se nenhum arquivo foi alterado. O formato da resposta, sozinho, não prova que a skill foi carregada.

Esse checklist é uma revisão inicial, não uma garantia de segurança. As permissões efetivas dependem da versão e da configuração; confira-as antes de tratar o cabeçalho do exemplo como bloqueio de edição. Documentação: https://code.claude.com/docs/en/skills

7 Planejar o gerador de orçamento

Escolha uma pasta onde você guarda seus projetos e abra o terminal nela. Agora será outro projeto; não crie a pasta dentro da landing por engano.

Terminal - criar e abrir o projeto

```
mkdir gerador-orcamento
cd gerador-orcamento
claude
```

Abra gerador-orcamento no editor. No Claude Code, pressione Shift+Tab até identificar o Plan Mode. Como alternativa, inicie a sessão pelo terminal com o comando abaixo:

```
claude --permission-mode plan
```

Permaneça em planejamento. Digite o pedido ou use o recurso de ditado do seu computador.

Conversa - pedido do projeto

```
Quero criar uma página web que me ajude a definir quanto cobrar por projetos de freelancer, de qualquer nicho. Eu descrevo o que o cliente precisa e posso fornecer uma referência, como um site, uma imagem ou um projeto parecido. A página analisa o escopo e, se faltar informação, me faz perguntas antes de estimar o preço. Depois, sugere uma faixa de valores e explica como chegou nela, considerando fatores como complexidade, prazo e entregáveis. Eu posso ajustar o valor e aprovar o valor final. Então, a página gera um link com a proposta para eu enviar ao cliente. O cliente abre o link, aprova a proposta e paga pelo Stripe.
```

Conversa - complementar a referência e as etapas

```
Mais duas coisas pro plano:

1. Todo orçamento sempre começa pedindo uma referência para se basear (um site, uma imagem ou um template que eu já tenho) e um campo onde eu escrevo o que vai ser aproveitado dessa referência para o cliente novo: quais seções e quais funcionalidades entram e quais ficam de fora. O cálculo do preço precisa levar em conta o que é reaproveitado e o que vai ser criado do zero.

2. Use este caso como exemplo no plano: um cliente quer uma landing page de academia. Eu já tenho um template pronto de academia. Vou aproveitar o topo com a chamada principal, a seção de planos e o formulário de contato; a loja de produtos não entra. Mostre como a página faria as perguntas e como chegaria na faixa de valores nesse caso.

Antes de fechar o plano, me faça as perguntas que faltarem. Divida o plano em etapas pequenas, começando pela versão mais simples que já funcione, e deixe a integração com o Stripe para uma etapa separada, começando no modo de teste.
```

Responda às perguntas do Claude. Forneça a imagem, o arquivo ou o endereço do seu template da academia quando necessário; a pauta não traz um link pronto para esse template.

Antes de aprovar

Confira se o fluxo pede a referência e o que será reaproveitado, faz perguntas quando falta informação, explica a faixa de preço e permite ajustar o valor. As etapas devem começar pela versão mais simples; Stripe deve ficar separado, em modo de teste. Peça ajustes até o plano refletir a sua intenção. A aula também cita Ctrl+G para editar o plano no editor, quando disponível.

8 Executar somente a primeira etapa

Depois de aprovar o plano, use o exemplo de /goal da aula se o comando estiver disponível na instalação. O pedido limita a execução à primeira etapa e define como apresentar o resultado.

Conversa - definir o objetivo

```
/goal a etapa 1 do plano aprovado está implementada: a página abre no navegador, aceita a descrição do projeto e a referência, faz pelo menos uma pergunta quando falta informação e mostra uma faixa de valores com a explicação; como prova, cole na conversa a saída do comando que abre ou serve a página e o texto completo da faixa de valores com a explicação do exemplo da academia; não comece a etapa 2; ou pare depois de 8 rodadas
```

Conversa - consultar o andamento

```
/goal
```

Conversa - cancelar o objetivo

```
/goal clear
```

Confira a página no navegador e as evidências na conversa. O exemplo pede parada após oito rodadas e não autoriza começar a etapa 2. Acompanhe o consumo e cancele quando necessário. Se /goal não estiver disponível, mantenha o plano salvo e confira a documentação antes de executar esse trecho: https://code.claude.com/docs/en/goal

9 Conferir a prática

[ ] A landing mostra os dois textos alterados no navegador.

[ ] As rules preservam as orientações do CLAUDE.md original.

[ ] O explicador respondeu no formato esperado sem editar o projeto.

[ ] A skill foi acionada e o relatório foi revisado.

[ ] O plano do gerador pede referência, perguntas e explicação do preço.

[ ] Se você executou o objetivo, a etapa 1 foi conferida e a etapa 2 não começou.

A seção de teste de leitura é temporária. Depois da conferência, você pode removê-la do arquivo em que ficou para deixar de receber a marca em toda resposta.

Exercício final

Escolha uma ideia sua, crie uma pasta vazia, entre no Plan Mode e explique o projeto. Revise o plano ao menos uma vez antes de aprovar. Se quiser repetir os exemplos, crie também um agente ou uma skill para essa nova ideia.

Comandos e atalhos citados na aula

claude --version: conferir a versão no terminal.

claude: abrir uma sessão no terminal da pasta do projeto.

/context: inspecionar o contexto carregado.

/doctor prompt-audit: revisão das instruções, quando disponível.

Shift+Tab: alternar os modos até o planejamento.

/plan antes de um pedido: solicitar planejamento, quando disponível.

Ctrl+G: abrir o plano no editor, quando disponível.

/checklist-seguranca-web: chamar a skill criada.

/goal: acompanhar o objetivo; /goal clear: cancelar.

Ctrl+O: expandir detalhes do registro, conforme a versão.

Fonte

Pauta da segunda aula, cópia local consultada em 14/10/2026. Todos os blocos de prompts e arquivos deste guia foram transcritos dessa pauta. O material não substitui a conferência do resultado no seu projeto.

https://docs.google.com/document/d/1Ci793TdKMeAEYHTENuy9n8Vi9IfCbs7w4D1vvgqfKWM/edit